Data Analytics & Optimization — Project Report

Team Info:

- 242000689 Mohamed Samy
- 231002196 Saray Youssef
- 231011495 Jana Emad
- 221001648 Mohamed Walid

Introduction

This report documents the complete solution developed for the PR 2021 – Product Market Type Classification competition hosted on Kaggle. The objective is to predict the **Market Type** (one of four classes: 0, 1, 2, 3) for retail products using a combination of product attributes and outlet characteristics.

The workflow follows the standard data science pipeline: exploratory analysis, data preprocessing, feature engineering, model selection, cross-validation, and generation of competition predictions.

Dataset Summary

Property	Training Set	Test Set
Rows	8,523	5,681
Features	10 (X1–X10)	10 (X1–X10)
Target	Y (classes 0–3)	— (to predict)
Missing: X2	1,463 (17.2%)	976 (17.2%)
Missing: X9	2,410 (28.3%)	1,606 (28.3%)

Feature Descriptions

Feature	Description	Type
X1	Product ID / Code	Object (dropped)
X2	Product Weight	Float64
X3	Fat Content	Object (ordinal)
X4	Product Visibility (shelf)	Float64
X5	Product Type / Category	Object (one-hot)
X6	Maximum Retail Price (MRP)	Float64
X7	Outlet ID	Object (one-hot)
X8	Outlet Establishment Year	Int64
X9	Outlet Size	Object (ordinal)

Feature	Description	Type
X10	Outlet Tier	Object → Int
Y	Market Type (target)	Int64

---

2. Exploratory Data Analysis

2.1 Missing Values

Two features contain missing values. **X2** (Product Weight) is missing for ~17% of rows; **X9** (Outlet Size) is missing for ~28%. No other features have missing data.

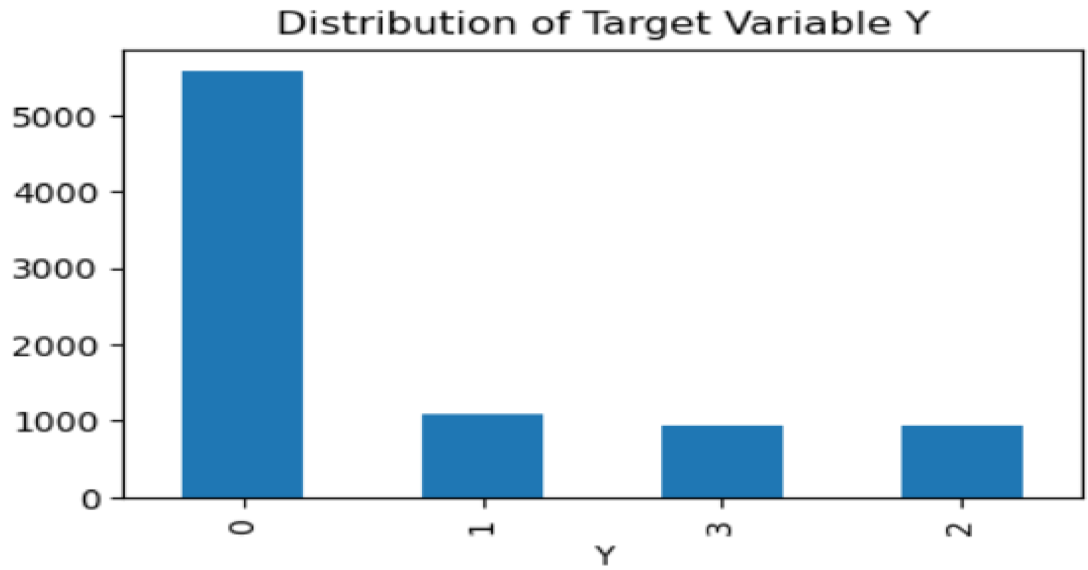
```
print("Missing values before handling:")
df.isnull().sum()
```

[10] ✓ 0.0s

```
Missing values before handling:

X2      1463
X3         0
X4         0
X5         0
X6         0
X7         0
X8         0
X9      2410
X10        0
Y          0
dtype: int64
```

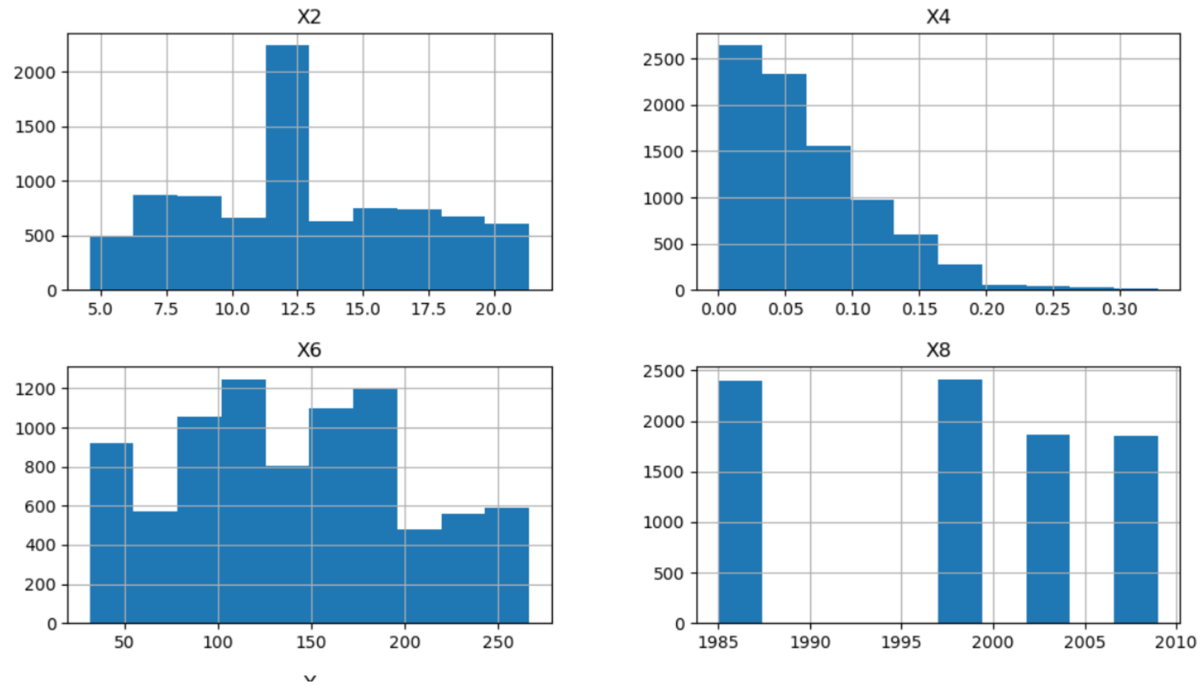
2.2 Target Variable – Class Imbalance

The target variable Y is heavily skewed: **Type 0** represents 65.4% of all training samples, while Types 1, 2, and 3 each account for roughly 11–13%. This imbalance is addressed in the final model using **class weights**.



2.3 Numeric Feature Distributions

Histograms of all numeric features reveal varying distribution shapes. Most features appear approximately normal or uniform, while **X4** (Product Visibility) exhibits strong right skew with a cluster of zero values.



3. Data Preprocessing

Label Inconsistency in X3 (Fat Content)

The Fat Content column (X3) contains 5 distinct labels that logically map to only 2 categories. *"low fat"* and *"LF"* are variants of **"Low Fat"**, while *"reg"* is a shortened form of **"Regular"**. This is standardized before encoding.

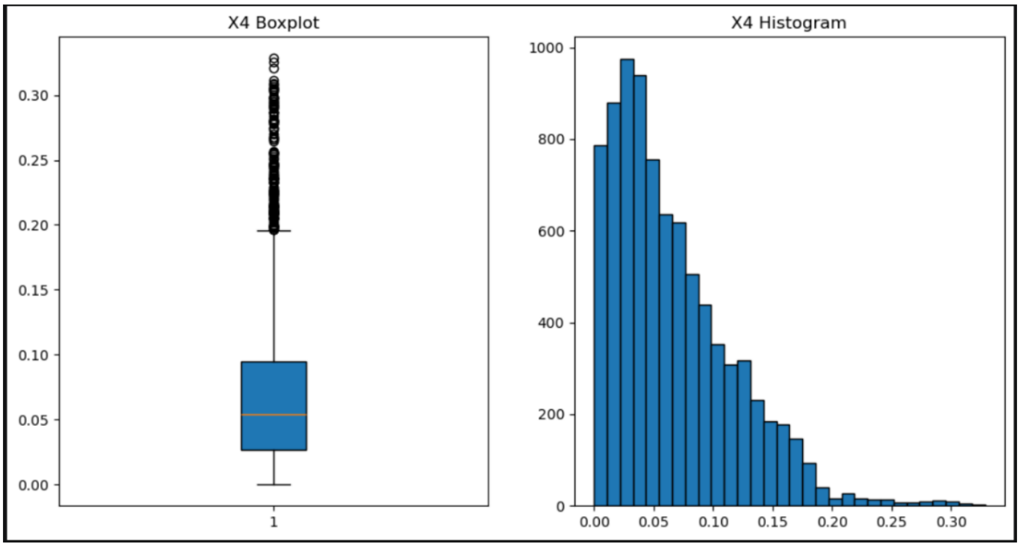
```python
# Display value counts for all categorical columns

categorical_cols = ['X3', 'X5', 'X7', 'X9']
for i in categorical_cols:
    print(f"Value counts for {i}:")
    print(df[i].value_counts())
    print("_"*25+"\n")
```

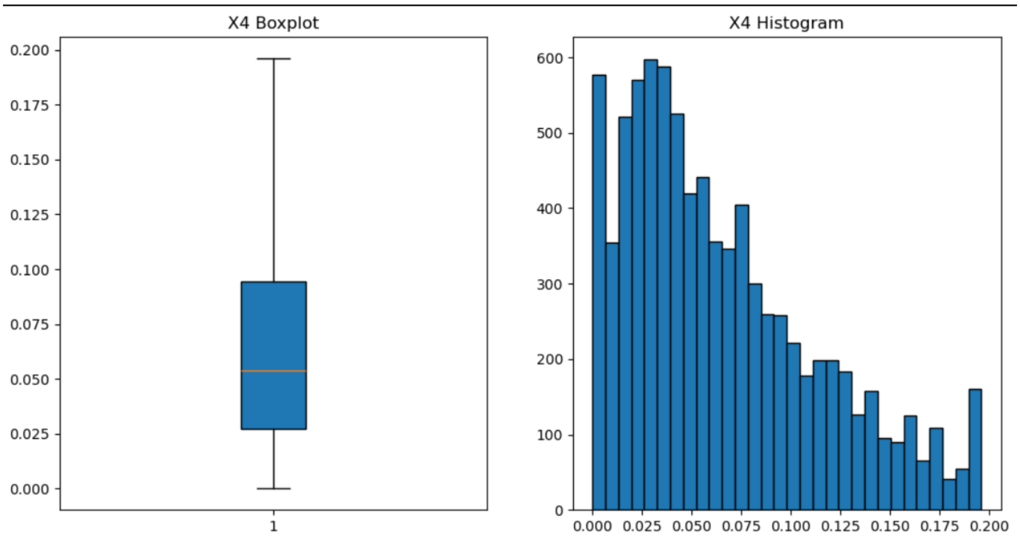
```
0.0s
```

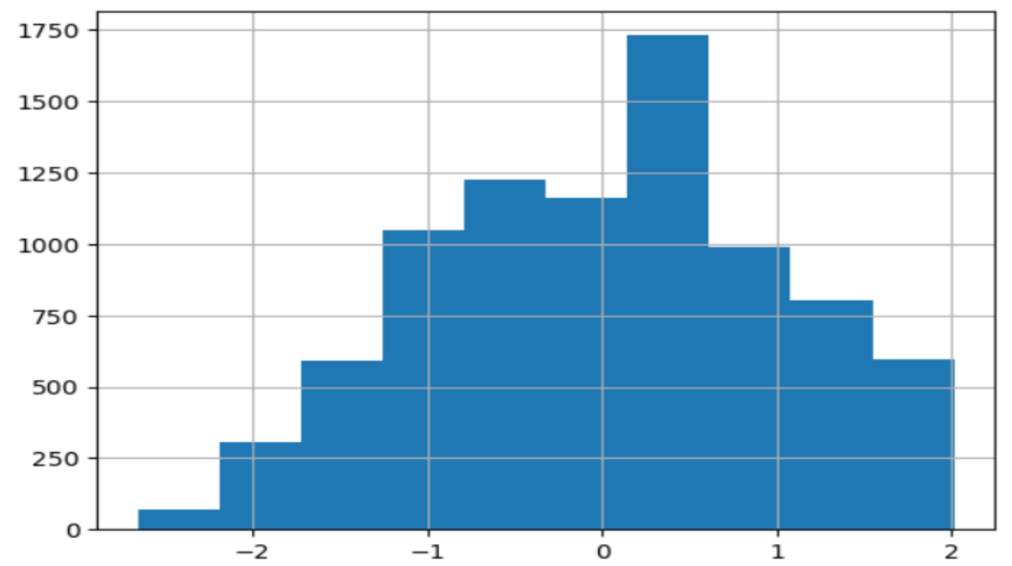
```
Value counts for X3:
X3
Low Fat     5089
Regular     2889
LF           316
reg          117
low fat      112
Name: count, dtype: int64
```

Outlier Handling in X4 (Product Visibility)

X4 is right-skewed and contains physical zero values (products with no shelf visibility). The IQR method is used to detect and clip outliers. Zero values are then replaced with the column mean before applying a **Box-Cox transformation** to reduce skewness.







distribution looks much better after Box-Cox transformation, with reduced skewness and fewer outliers.

4. Modelling & Evaluation

Baseline – Gaussian Naïve Bayes

Naïve Bayes serves as a probabilistic baseline. It assumes feature independence and models each class as a Gaussian distribution over feature values.

```python
# Modeling with Gaussian Naive Bayes

nb_classifier = GaussianNB()
nb_classifier.fit(X_val, y_val)

# Make predictions
y_pred = nb_classifier.predict(X_val)



# Evaluate the model

acc = accuracy_score(y_val, y_pred)

f1 = f1_score(
    y_val,
    y_pred,
    average='macro'
)

print("Accuracy:", f"{acc:.2f}")
print("Macro F1 Score:", f"{f1:.2f}")
```
✓ 0.0s

```
Accuracy: 0.83
Macro F1 Score: 0.71
```

Random Forest

An ensemble of 200 decision trees trained with bagging. Each tree votes on the predicted class; the majority vote determines the final prediction. Random Forest naturally handles non-linear relationships and provides feature importance rankings.

```python
# Modeling with Random Forest Classifier

model = RandomForestClassifier(
    n_estimators=200,
    random_state=42
)

model.fit(X_train, y_train)

y_pred = model.predict(X_val)



acc = accuracy_score(y_val, y_pred)

f1 = f1_score(
    y_val,
    y_pred,
    average='macro'
)

print("Accuracy:", f"{acc:.2f}")
print("Macro F1 Score:", f"{f1:.2f}")
```

✓ 0.9s

```
Accuracy: 0.93
Macro F1 Score: 0.85
```

CatBoost with Native Categorical Features

CatBoost is a gradient boosting framework that processes categorical features internally using ordered target statistics, eliminating the need for manual one-hot encoding. This allows X5 and X7 to be passed directly as strings.

Key Finding: Outlet ID as Primary Predictor

Exploratory analysis reveals that **Outlet ID (X7)** is the dominant predictor. Each outlet maps almost exclusively to a single market type, meaning the market type is effectively an outlet-level attribute rather than a product-level one. This explains the very high accuracy achieved by tree-based models.

Prediction & Submission

The final model is trained on the **entire training dataset** (no held-out split) using the best CatBoost configuration found during cross-validation. Predictions are generated for the test set and saved in the required submission format.

```python
# Build and save the submission CSV in the required format
submission = pd.DataFrame({
    'row_id': range(len(test_X)),
    'label': test_preds.astype(int)
})

submission.to_csv('submission.csv', index=False)
```

0.0s

```python
submission.head()
```

0.0s

	row_id	label
0	0	0
1	1	0
2	2	1
3	3	0
4	4	3

Kaggle Submission Score:

Kaggle Score: 0.99251

YOUR RECENT SUBMISSION

submission.csv

Submitted by Mohamed Samy · Submitted 2 minutes ago

Score: 0.99251

Private score: 0.98885

Jump to your leaderboard position

s

Conclusion

This project successfully built a multiclass classification pipeline for the PR 2021 Market Type competition. Key takeaways:

- **Data quality:** Two features had significant missing data (X2: 17%, X9: 28%), and X3 had 5 inconsistent labels that required standardization before encoding.
- **Feature transformation:** X4 (Product Visibility) required IQR outlier clipping, zero-value imputation, and Box-Cox transformation to reduce right skew.
- **Class imbalance:** Type 0 accounts for 65% of training samples. This was addressed using class_weights in CatBoost rather than resampling.
- **Model selection:** CatBoost with native categorical handling significantly outperformed Naïve Bayes (F1: 70.3%) and Random Forest (F1: 84.9%), achieving a 5-Fold CV Macro F1 of 98.9%.
- **Key insight:** Outlet ID (X7) is the dominant feature, as each outlet maps almost exclusively to a single market type.